

```
from google.colab import drive
drive.mount('/content/drive')
```

```
-----
MessageError                                Traceback (most recent call last)
/tmp/ipykernel_2073/1408506528.py in <cell line: 0>()
      1 from google.colab import drive
----> 2 drive.mount('/content/drive')

3 frames
/usr/local/lib/python3.12/dist-packages/google/colab/_message.py in read_reply_from_input(message_id, timeout_sec)
    101 ):
    102     if 'error' in reply:
--> 103         raise MessageError(reply['error'])
    104     return reply.get('data', None)
    105

MessageError: Error: credential propagation was unsuccessful
```

Next steps: [Explain error](#)

Alternative to `drive.mount()`: Using `PyDrive`

If `drive.mount()` is not working or if you prefer to interact with Google Drive files programmatically rather than mounting it as a filesystem, you can use the `PyDrive` library. This allows you to authenticate and then list, upload, or download files directly.

```
# Install the PyDrive library
!pip install -U -q PyDrive
```

```
987.4/987.4 kB 3.9 MB/s eta 0:00:00
Preparing metadata (setup.py) ... done
Building wheel for PyDrive (setup.py) ... done
```

```
from pydrive.auth import GoogleAuth
from pydrive.drive import GoogleDrive
from google.colab import auth
from oauth2client.client import GoogleCredentials

# Authenticate Colab user (this will open a new window for authentication if needed)
auth.authenticate_user()

# Create GoogleAuth instance.
gauth = GoogleAuth()

# Explicitly set 'embedded' to True to prevent PyDrive from attempting
# to launch a local webserver for authentication, which requires client_secrets.json.
# This forces PyDrive to use an authentication flow suitable for Colab.
gauth.embedded = True

# Explicitly set 'client_config_backend' in gauth's settings AFTER initialization
# as a defensive measure against other potential client config loads.
gauth.settings['client_config_backend'] = 'settings'

# Load the credentials obtained from google.colab.auth
gauth.credentials = GoogleCredentials.get_application_default()

# Create the PyDrive client using the gauth object
drive = GoogleDrive(gauth)

print('PyDrive client authenticated and ready.')
```

```
-----
TypeError                                Traceback (most recent call last)
/tmp/ipykernel_2073/1264159616.py in <cell line: 0>()
    18 # Create the PyDrive client using the authorized HTTP object
    19 # This bypasses GoogleAuth's internal flow that was looking for client_secrets.json
----> 20 drive = GoogleDrive(http=http_auth)
    21
    22 print('PyDrive client authenticated and ready using direct http object.')
```

TypeError: GoogleDrive.__init__() got an unexpected keyword argument 'http'

Next steps: [Explain error](#)

```
# Example: List files in your Google Drive root directory
# You can adjust the 'q' parameter to search for specific files/folders
file_list = drive.ListFile({'q': "'root' in parents and trashed=false"}).GetList()
for file in file_list:
    print('title: %s, id: %s' % (file['title'], file['id']))

print(f'\nFound {len(file_list)} files/folders in your Drive root.')
```

Enal
noti
to g
exer

```
-----
FileNotFoundError                                Traceback (most recent call last)
/usr/local/lib/python3.12/dist-packages/oauth2client/clientsecrets.py in _loadfile(filename)
    120     try:
--> 121         with open(filename, 'r') as fp:
    122             obj = json.load(fp)
```

FileNotFoundError: [Errno 2] No such file or directory: 'client_secrets.json'

During handling of the above exception, another exception occurred:

```
InvalidClientSecretsError                        Traceback (most recent call last)
-----
      10 frames -----
```

InvalidClientSecretsError: ('Error opening file', 'client_secrets.json', 'No such file or directory', 2)

During handling of the above exception, another exception occurred:

```
InvalidConfigError                              Traceback (most recent call last)
/usr/local/lib/python3.12/dist-packages/pydrive/auth.py in LoadClientConfigFile(self, client_config_file)
    386     client_type, client_info = clientsecrets.loadfile(client_config_file)
    387     except clientsecrets.InvalidClientSecretsError as error:
--> 388         raise InvalidConfigError('Invalid client secrets file %s' % error)
    389     if not client_type in (clientsecrets.TYPE_WEB,
    390                           clientsecrets.TYPE_INSTALLED):
```

InvalidConfigError: Invalid client secrets file ('Error opening file', 'client_secrets.json', 'No such file or directory', 2)

Next steps: [Explain error](#)

```
from google.colab import drive
drive.mount('/content/drive', force_remount=True)

from google.colab import files
uploaded = files.upload()

import pandas as pd
import io
df = pd.read_csv(io.BytesIO(list(uploaded.values())[0]))
print(df.shape)
```

Mounted at /content/drive

No file chosen

```
-----
KeyboardInterrupt                                Traceback (most recent call last)
/tmp/ipykernel_2073/965394466.py in <cell line: 0>()
     4
     5 from google.colab import files
--> 6 uploaded = files.upload()
     7
     8
```

```
-----
      3 frames -----
/usr/local/lib/python3.12/dist-packages/google/colab/_message.py in read_reply_from_input(message_id, timeout_sec)
    94     reply = _read_next_input_message()
    95     if reply == _NOT_READY or not isinstance(reply, dict):
--> 96         time.sleep(0.025)
    97         continue
    98     if (
```

KeyboardInterrupt:

```
from google.colab import drive
drive.mount('/content/drive', force_remount=True)
```

Mounted at /content/drive

```
from google.colab import files
uploaded = files.upload()
```

tested.csv
tested.csv(text/csv) - 29474 bytes, last modified: 6/16/2026 - 100% done
Saving tested.csv to tested (2).csv

```
import pandas as pd
import io
df = pd.read_csv(io.BytesIO(list(uploaded.values())[0]))
print(df.shape)
```

(418, 12)

```
print(df.columns)
```

```
Index(['PassengerId', 'Survived', 'Pclass', 'Name', 'Sex', 'Age', 'SibSp',
       'Parch', 'Ticket', 'Fare', 'Cabin', 'Embarked'],
      dtype='object')
```

Q#1 Definitions

```
# 1. NumPy
print("1. NumPy:")
print("NumPy is a Python library used for numerical computations. It supports arrays, matrices, and mathematical functions.")
print()

# 2. Matplotlib
print("2. Matplotlib:")
print("Matplotlib is a Python library used for data visualization. It creates charts, graphs, and plots.")
print()

# 3. Pandas
print("3. Pandas:")
print("Pandas is a Python library used for data manipulation and analysis. It works with tables called DataFrames.")
print()

# 4. TensorFlow
print("4. TensorFlow:")
print("TensorFlow is a Python library by Google used for deep learning and building neural networks.")
print()

# 5. Seaborn
print("5. Seaborn:")
print("Seaborn is a Python library for statistical data visualization. It makes beautiful charts easily.")
print()

# 6. Sklearn
print("6. Sklearn (Scikit-learn):")
print("Sklearn is a Python library for machine learning. It has tools for classification, regression, and clustering.")
```

```
1. NumPy:
NumPy is a Python library used for numerical computations. It supports arrays, matrices, and mathematical functions.

2. Matplotlib:
Matplotlib is a Python library used for data visualization. It creates charts, graphs, and plots.

3. Pandas:
Pandas is a Python library used for data manipulation and analysis. It works with tables called DataFrames.

4. TensorFlow:
TensorFlow is a Python library by Google used for deep learning and building neural networks.

5. Seaborn:
Seaborn is a Python library for statistical data visualization. It makes beautiful charts easily.

6. Sklearn (Scikit-learn):
Sklearn is a Python library for machine learning. It has tools for classification, regression, and clustering.
```

```
# Q#2 Dataset Analysis

# Number of features (columns)
print("Number of Features (columns):", df.shape[1])
print()

# Number of instances (rows)
print("Number of Instances (rows):", df.shape[0])
print()

# List at least 5 features
print("5 Features of Titanic Dataset:")
print("1.", df.columns[0])
print("2.", df.columns[1])
print("3.", df.columns[2])
print("4.", df.columns[3])
print("5.", df.columns[4])
```

Number of Features (columns): 12

Number of Instances (rows): 418

5 Features of Titanic Dataset:

1. PassengerId
2. Survived
3. Pclass
4. Name
5. Sex

```
# Dataset Information
```

```
print("1. Website Name of Dataset:")
print("   Kaggle - www.kaggle.com")
print()

print("2. Is Titanic a Benchmark Dataset?")
print("   YES! Titanic is a very famous Benchmark dataset.")
print("   It is used worldwide for learning Machine Learning.")
print()

print("3. Is data Structured or Unstructured?")
print("   STRUCTURED data - because it is in table format (rows and columns)")
print()

print("4. Check if any Text in cells:")
print(df.dtypes)
```

```
1. Website Name of Dataset:
   Kaggle - www.kaggle.com

2. Is Titanic a Benchmark Dataset?
   YES! Titanic is a very famous Benchmark dataset.
   It is used worldwide for learning Machine Learning.

3. Is data Structured or Unstructured?
   STRUCTURED data - because it is in table format (rows and columns)

4. Check if any Text in cells:
PassengerId    int64
Survived       int64
Pclass         int64
Name           object
Sex            object
Age            float64
SibSp          int64
Parch          int64
Ticket         object
Fare           float64
Cabin          object
Embarked       object
dtype: object
```

```
# ML Process Steps
```

```
print("Machine Learning Process:")
print()
print("1. Data Collection")
print("2. Data Preprocessing")
print("3. Exploratory Data Analysis (EDA)")
```

Enal
noti
to g
exer

```
print("4. Feature Selection")
print("5. Model Selection")
print("6. Model Training")
print("7. Model Testing")
print("8. Model Evaluation")
print("9. Model Deployment")
```

Machine Learning Process:

1. Data Collection
2. Data Preprocessing
3. Exploratory Data Analysis (EDA)
4. Feature Selection
5. Model Selection
6. Model Training
7. Model Testing
8. Model Evaluation
9. Model Deployment

```
# Prepare data for KNN Model
from sklearn.preprocessing import LabelEncoder

# Drop text columns that can't be used
df_model = df.drop(['Name', 'Ticket', 'Cabin'], axis=1)

# Fill missing values
df_model = df_model.fillna(df_model.mean(numeric_only=True))
df_model['Embarked'] = df_model['Embarked'].fillna('S')

# Convert text to numbers
le = LabelEncoder()
df_model['Sex'] = le.fit_transform(df_model['Sex'])
df_model['Embarked'] = le.fit_transform(df_model['Embarked'])

print("Data Prepared Successfully!")
print(df_model.shape)
```

Data Prepared Successfully!
(418, 9)

```
# Train KNN Model
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier

# Define X (features) and y (target)
X = df_model.drop('Survived', axis=1)
y = df_model['Survived']

# Split data into training and testing
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Create KNN model
knn = KNeighborsClassifier(n_neighbors=5)

# Train the model
knn.fit(X_train, y_train)

print("KNN Model Trained Successfully!")
print("Training size:", X_train.shape)
print("Testing size:", X_test.shape)
```

KNN Model Trained Successfully!
Training size: (334, 8)
Testing size: (84, 8)

```
# Testing and Confusion Matrix
from sklearn.metrics import confusion_matrix, accuracy_score, classification_report

# Test the model
y_pred = knn.predict(X_test)

# Training Result
train_score = knn.score(X_train, y_train)
print("Training Result:", round(train_score * 100, 2), "%")

# Testing Result
test_score = accuracy_score(y_test, y_pred)
```

```
print('Testing Result: ', round(test_score * 100, 2), '%')
print()

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:")
print(cm)
print()

# Detailed Report
print("Classification Report:")
print(classification_report(y_test, y_pred))
```

Training Result: 76.65 %
Testing Result: 63.1 %

Confusion Matrix:
[[42 8]
 [23 11]]

Classification Report:

	precision	recall	f1-score	support
0	0.65	0.84	0.73	50
1	0.58	0.32	0.42	34
accuracy			0.63	84
macro avg	0.61	0.58	0.57	84
weighted avg	0.62	0.63	0.60	84

Enal
noti
to g
exec